RegNo: 222008181 **On 15th March 2026**

UNIVERSITY OF RWANDA

Dept: CSE

Mobile Application Systems and Design (#Lab 4)

Uwizeyimana Gaetan
(222008181)
CSE

On 15th March 2026

Mobile Application Systems and Design #LAB4

1. Which Dependencies did you use and why?

> Flutter (SDK + Material): It was used to build whole UI of the Posts Manager app (Screens, buttons, cards, forms, navigation and theming).

> http: It was used to call the JSONPlaceholder API provided and used for all Http methods (GET to list posts, POST to create posts, PUT to edit posts and DELETE to remove posts).

> flutter_launcher_icons: Was used to generate a custom android launcher icon from Assets/logo.png and to make installed app show the posts Manager logo and name in the phone's app drawer.

2. Generally, how can you handle API exceptions (network errors, invalid data)?

> Wrap API calls in try/catch: Surround each http.get/Post/put/delete with try and catch exception types so the app does not crash.

> Check Http status codes: Treat only 200 (or 201 for create) as success. For other codes (e.g. 400, 403, 500) throw an error or handle specially (in my case 403 shows fall back sample posts)

> Handle network-specific errors: Catch SocketException to detect no internet connection or server unrachable. Catch TimeoutException (if using timeouts) when the server is too slow.

> Handle invalid or unexpected data: Catch FormatException when jsonDecode fails or the JSON structure is not what the app expects. Optionally validate required fields (like title and body) before using them.

> Reflect errors in the UI instead of crashing:

* Use FutureBuilder to show; a loading spinner while waiting, an error message + Retry button when there is an exception, normal content when data is loaded.

* Show SnackBar messages for failures on create, update or delete operation

> Optional graceful fallback: When the real API is blocked (e.g HTTP 403), return local sample data so that the app can still demonstrate the UI and functionality.

3. Explain the future widget you have used, and the part of the UI that depends on the Future.

> I used FutureBuilder <List<post>> in the postsList Screen. The Future inside the FutureBuilder is _Service.fetchPosts(), which returns a Future<List<Post>>.

The posts list UI depends on this Future:

* When ConnectionState is waiting, the Screen shows a loading spinner in the centre

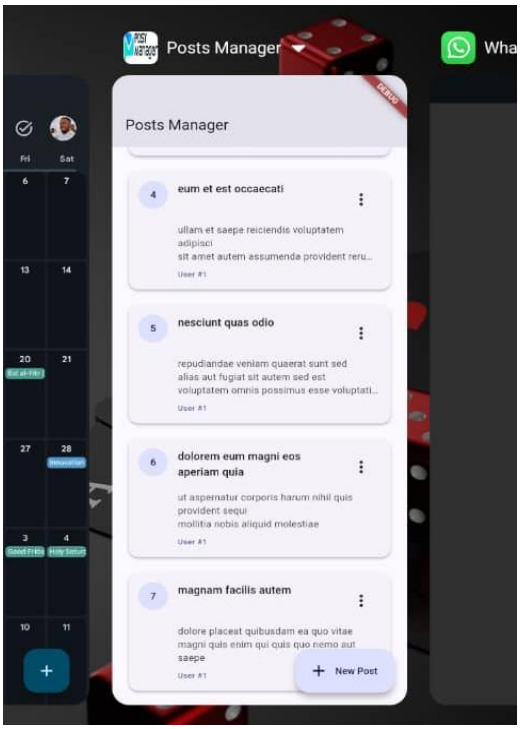
* When the Future completes with data, the screen shows a scrollable list of posts using cards (title, bodypreview, userid, actions).

* When the Future completes with an error (for example HTTP 403 or no internet), the screen shows an error message and a retry button.

This demonstrates that part of the UI is reactive to asynchronous API calls and updates automatically when the Future finishes.

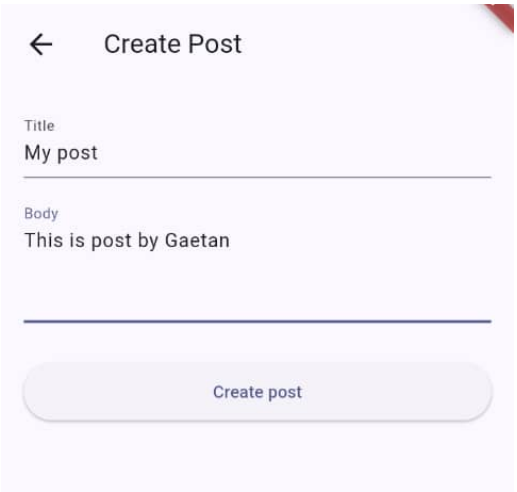
Screenshots:

1. App Look and fill:

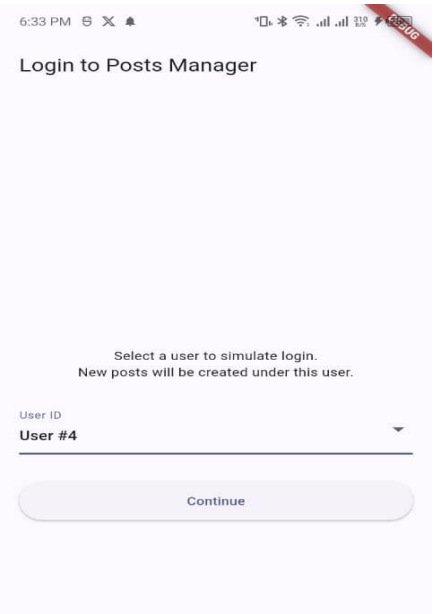


2. Content retrieval:

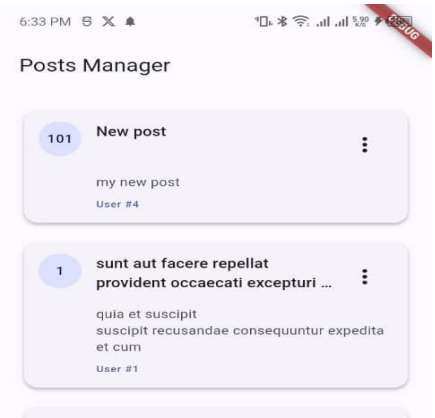
3.Creating Post:



4. User selection (Considered as session activation to get id of user):



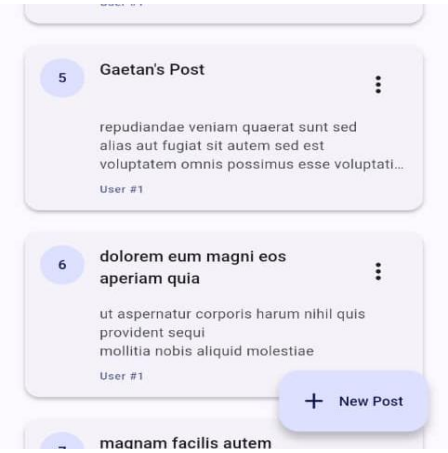
5. Created post:



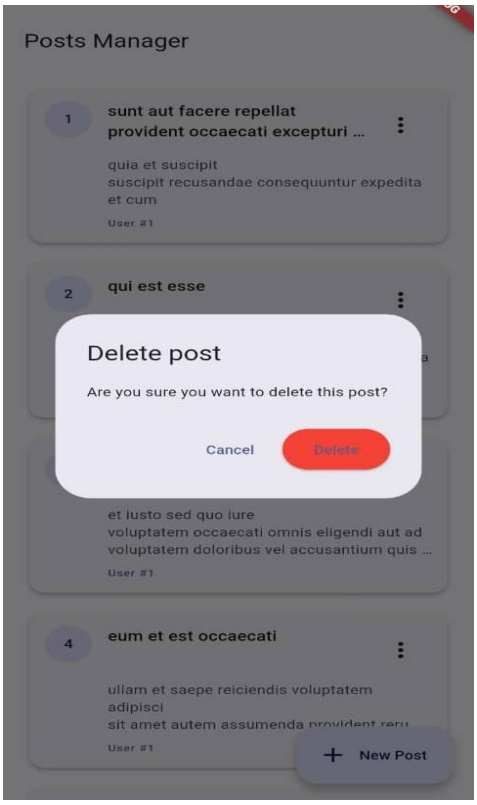
6. Post update/delete menu:

7.Post editing:

8.Edited Post:



9. Deleting Post:



******** For Better screenshots view, Look into assets!! ********